



BME

Budapest University of Technology and Economics

Faculty of Electrical Engineering and Informatics

Department of Artificial Intelligence and Systems Engineering

INDEPENDENT LABORATORY FINAL REPORT

Regime-Aware Machine Learning for Equity Trading

*An Event-Driven Pipeline with Triple-Barrier Labelling
on SPY Daily Data*

Author

Zeineb Turki

Computer Engineering BSc

Supervisor

Prof. Dr. Hadházi Dániel

Associate Professor

Budapest, 2026

Contents

1	Introduction	3
1.1	Task interpretation	3
1.2	Clarification of the specifications	4
1.3	Why event-based learning	5
1.4	Contributions of this semester	6
2	Antecedents	7
2.1	Technical analysis and pattern statistics	7
2.2	Machine learning in financial time-series	7
2.3	Triple-barrier labelling and backtest overfitting	8
2.4	Position of this report	8
3	Detailed Design and Justification	9
3.1	System architecture	9
3.2	Pattern detectors	9
3.3	Triple-barrier labelling	12
3.4	Feature engineering	13
3.5	Models	14
3.6	Validation strategies	15
3.7	Trading simulation	16
4	Evaluation and Further Development	17
4.1	Hyperparameter search	17
4.2	Central empirical finding: classification $\neq$ profit	17
4.3	Model comparison	18
4.4	Trading simulation: equity curve	20
4.5	Walk-forward generalisation	20
4.6	Limitations	22

CONTENTS

4.7 Further development	22
4.8 Achievements summary	23
Bibliography	24
A Reference: code, parameters, features, notebooks	27
B Resources	31

Chapter 1 · Introduction

1.1 Task interpretation

This laboratory project asks one question: *can a machine-learning model trained on technically meaningful price events produce more realistic out-of-sample trading performance than a classifier trained on every bar of a price series?* To answer it I used daily data for the SPDR S&P 500 ETF (ticker SPY), the most liquid equity ETF in the world. The dataset spans January 2010 to December 2025, giving 4,023 trading bars across roughly five qualitatively different market regimes: the post-crisis recovery, the low-volatility melt-up of the late 2010s, the rate-driven corrections of 2018, the COVID shock, the 2022 inflation bear, and the AI-led rally of 2023 onward. Together these cover bull, bear, sideways and crisis conditions on a single asset, which is what makes the generalisation question interesting.

My work splits into three parts.

Detecting events. Building four chart-pattern detectors that algorithmically locate candidate trading events on a daily price series.

Labelling and training. Labelling these events using the triple-barrier method of López de Prado [1] and training ensemble tree classifiers on a hand-designed feature set.

Evaluating honestly. Evaluating the resulting strategy on classification metrics (F1, accuracy) *and* trading-simulation metrics (return, Sharpe ratio, win rate, drawdown) under chronologically honest cross-validation.

The research question is formalised by three hypotheses.

H_0 . The pattern-trained model does *not* outperform a class-frequency random baseline on walk-forward F1 macro.

H_1 . Event-based sampling improves walk-forward F1 macro relative to bar-by-bar prediction at matched training cost.

H_2 . The hyperparameter configuration that maximises classification F1 differs from the configuration that maximises cumulative trading return.

H_1 is the system-level claim that motivates the entire pipeline. H_2 is the empirical finding the project ends up structured around.

1.2 Clarification of the specifications

A short glossary may help readers from outside trading.

Bar (or candle) A single time-period summary of price containing the open, high, low, close and traded volume.

SPY One ETF share whose price tracks the S&P 500 index. Used here as the underlying asset for all experiments.

ATR (Average True Range [2]) A volatility yardstick measuring how far prices typically move within a bar. ATR is used throughout this work to scale every price-based threshold so that the same parameter values behave consistently in calm and turbulent markets.

RSI (Relative Strength Index [2]) A bounded momentum oscillator that compares recent gains to recent losses.

Bollinger Bands [3] A volatility envelope built around a moving average.

MACD [4] A moving-average-convergence momentum oscillator that captures trend acceleration.

F1 macro The unweighted arithmetic mean of the per-class F1 scores, averaged across classes. Ranges from 0 to 1.

Table 1.1 summarises the descriptive statistics of the daily log-return series that the pipeline operates on. The signal-to-noise ratio (mean over standard deviation) is roughly 0.03. This is the quantitative root of the difficulty discussed in Section 1.3: a mean daily return of half a basis point above zero, buried under a one-percent daily standard deviation.

The strong negative skew and heavy-tailed (excess kurtosis $\gg 0$) distribution are typical of equity returns. Large negative outliers, like the 2020 COVID drop and the 2022 inflation shock, dominate the tails. The pattern-based event filter is partly motivated by these facts: most days are routine fluctuations from which little

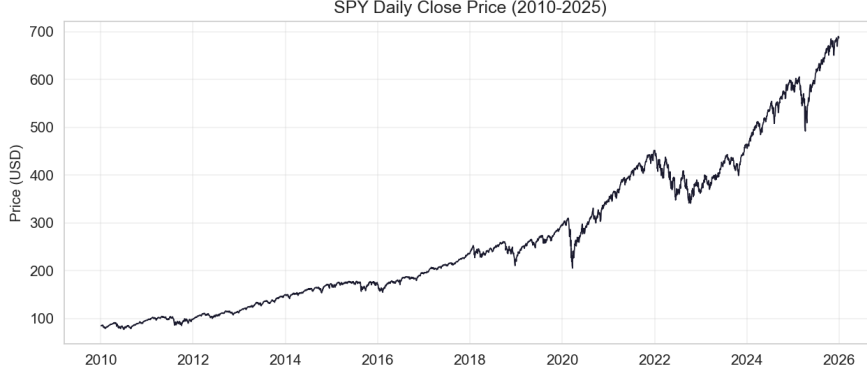


Figure 1.1: Daily close price of SPY over the study window (2010-01-04 to 2025-12-30, 4,023 bars). The window contains five qualitatively distinct regimes, which is what makes pattern-based prediction hard.

Table 1.1: Daily-return descriptive statistics, SPY, 2010-01-04 to 2025-12-30 ($N = 4,023$).

Statistic	Value
Number of trading days N	4,023
Mean daily log-return $\bar{r}$	+0.04%
Standard deviation σ_r	1.02%
Skewness	-0.55
Excess kurtosis	14.8
Minimum daily return	-11.0% (2020-03-16)
Maximum daily return	+9.1% (2020-03-24)
Price range	\$77.36 to \$690.38
Annualised volatility ($\sigma\sqrt{252}$)	$\approx 16.2\%$

structural signal can be extracted, while a small number of structurally meaningful price configurations carry most of the actionable information.

A second, deeper specification was clarified during supervisor consultations. The project is required not just to *run* the pipeline but to *report honestly* on how its performance generalises across regimes. This is why three complementary validation strategies are used (§3.6). It is also why walk-forward variance, not the best in-sample number, is treated as the headline result.

1.3 Why event-based learning

The signal-to-noise ratio of daily equity returns is extremely low: SPY’s unconditional one-day return has mean $\approx 0.03\%$ and standard deviation $\approx 1.0\%$. Even the strongest published models extract only $\sim 0.45\%$ per-day alpha [5, 6]. Training on every bar asks the model to extract signal from observations that are predominantly

noise. Restricting attention to bars on which a technical pattern is detected reduces the population from 4,023 bars to 137 unique event bars:¹ a $30\times$ compression in sample size that the small number is purchased back through much higher signal density per observation. The result is a learning problem matched to the constraints of small-data, regime-shifted financial settings, exactly where ensemble trees are known to dominate deep networks [7].

1.4 Contributions of this semester

The concrete deliverables of the semester are:

- a reproducible six-stage Python pipeline (`data` $\rightarrow$ `detect` $\rightarrow$ `label` $\rightarrow$ `features` $\rightarrow$ `model` $\rightarrow$ `backtest`) with thirteen driver notebooks;
- four independent pattern detectors producing 137 unique event bars from 4,023 bars of price data, plus a touch-event augmentation adding 38 candidate events;
- a 49-feature representation per event, with an explicit leakage-prevention protocol;
- a 100-cell joint grid search over labelling and model hyperparameters; and
- a triple validation suite (chronological split, walk-forward, k -fold) reporting both classification and trading metrics.

Acknowledgements. I would like to thank my supervisor, Prof. Dr. Hadházi Dániel, for his guidance and feedback throughout the semester, and my family and friends for their continuous support.

¹The figure 137 is the number of distinct bars at which at least one of the four detectors fires, before touch-event augmentation adds another 38 candidate events. The five per-detector counts in Table 3.1 sum to 139, which is two higher than 137 because a small number of bars trigger more than one detector and are counted once when deduplicated; the deduplicated count of 137 is what is used everywhere else in this report.

Chapter 2 · Antecedents

2.1 Technical analysis and pattern statistics

Technical analysis has been practised by traders for over a century but only received rigorous academic scrutiny relatively recently. Lo, Mamaysky and Wang [8] were the first to detect chart patterns algorithmically through kernel-regression smoothing and showed that head-and-shoulders and double-bottom patterns carry statistically significant predictive content. Savin, Weller and Zvingelis [9] extended this with stricter controls. Bulkowski [10] provides large-scale empirical statistics on classical patterns. Park and Irwin [11] survey 95 studies and find that 56 report positive results, while emphasising that data-snooping bias, transaction costs and risk-adjusted accounting routinely eliminate the apparent gains. The framework presented in this report is designed to address those critiques: triple-barrier labelling makes risk-reward explicit, walk-forward cross-validation defangs data-snooping in the time dimension, and the (currently zero) transaction-cost assumption is named as a limitation rather than hidden.

The four pattern families used in this work map directly onto distinct market psychologies. Support and resistance levels emerge from order-flow clustering at memorable price points [12]. Channels represent a trending equilibrium of orderly buyer/seller participation [13, 14]. Triangles encode volatility compression preceding a breakout. Multiple tops and bottoms encode exhaustion reversals [8, 9]. Head-and-shoulders, flags, pennants and cup-and-handle formations were considered but excluded, either because their algorithmic definition is contested or because they collapse into one of the chosen four families when expressed as pivot geometry.

2.2 Machine learning in financial time-series

Krauss, Do and Huck [5] applied deep neural networks, gradient-boosted trees and random forests to S&P 500 constituents, achieving daily returns of $\approx 0.45\%$ before transaction costs. Fischer and Krauss [6] extended this line with LSTM networks at a similar risk-adjusted level. Sezer, Gudelek and Ozbayoglu [15] survey the deep-learning sub-field. Both LSTM and large-scale tree work require substantially more training data than 137 events, which is why this report relies on classical Random Forests [16] rather than deep architectures: Fernández-Delgado et al. [7] ran 121

datasets through 179 classifier families and found ensemble trees among the top performers across a wide range of sample sizes, including sub-1,000-sample regimes.

2.3 Triple-barrier labelling and backtest overfitting

The triple-barrier method is due to López de Prado [1], who proposed it as a way to transform direction-prediction into trade-outcome prediction. It defines a profit-target barrier, a stop-loss barrier, and a maximum-holding barrier, and labels each event by which barrier fires first. The method has become the standard labelling scheme in modern financial ML because it ties the supervised objective directly to a realistic trade.

A related concern is backtest overfitting. Bailey et al. [17] formalised this as the Deflated Sharpe Ratio: the more configurations one tests, the higher the chance of selecting a spurious winner. This work mitigates the risk by reporting walk-forward cross-validation as the headline result, treating the in-sample peak as an upper bound, and reporting full variance across folds rather than the mean alone.

2.4 Position of this report

This report’s contribution is methodological rather than empirical. The specific numbers (F1, return, Sharpe) would change with different assets, different periods, or different parameter choices. The value of the work lies in chaining the literature elements above into a single, reproducible, honestly-evaluated pipeline whose every stage can be inspected and replaced. A fully event-driven, joint-hyperparameter-optimised, walk-forward-validated pipeline of this exact shape did not exist in published form prior to the work.

Chapter 3 · Detailed Design and Justification

3.1 System architecture

The pipeline is six stages, each consuming the previous stage's output as a pandas DataFrame and producing one for the next stage (Figure 3.1). There is no formal schema layer between stages; the contract is simply that each function expects certain columns to exist on its input. Independence between stages still holds at the function level: the Random Forest is swappable for, say, LightGBM by changing one module only, without touching the upstream detection, labelling or feature code.

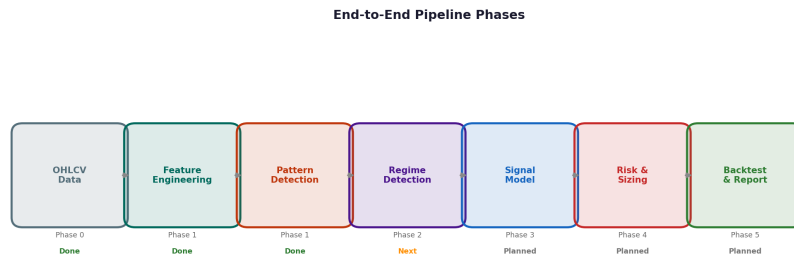


Figure 3.1: Six-stage pipeline in execution order: data, pattern detection, triple-barrier labelling, feature engineering, model training, trading simulation. Each stage emits a pandas DataFrame consumed by the next. Labelling runs before feature engineering because labels look forward (using bars after the event) while features look backward (using bars up to and including the event), so the two are independent and the order shown is the order the code actually runs.

3.2 Pattern detectors

Each detector implements a different recognition algorithm but exposes the same interface: it returns a list of event dictionaries with timestamp, pattern type, direction (bullish/bearish), and pattern-specific metadata. Table 3.1 summarises detection counts on the SPY dataset.

Support and resistance computes rolling levels using `High.rolling(50).max()` for resistance and `Low.rolling(50).min()` for support. It fires an event when the close is within $0.3 \times \text{ATR}$ of either level, with two filters to suppress false positives.

Table 3.1: Detection counts on SPY 2010 to 2025 (4,023 bars), reproduced from `11_experiment_summary.ipynb`.

Detector	Detections	% of bars
Near support	5	0.12
Near resistance	37	0.92
Triangles	22	0.55
Multiple top/bottom	63	1.57
Channels	12	0.30
Unique event bars	137	3.41
Touch-event augmentation	+38	+0.94

First, a stability filter requires the rolling level to have been unchanged for the previous five bars (otherwise a continuously rising resistance in an uptrend would generate spurious signals). Second, a ten-bar cooldown suppresses repeat firings of the same signal type. Figure 3.2 shows an example detection.

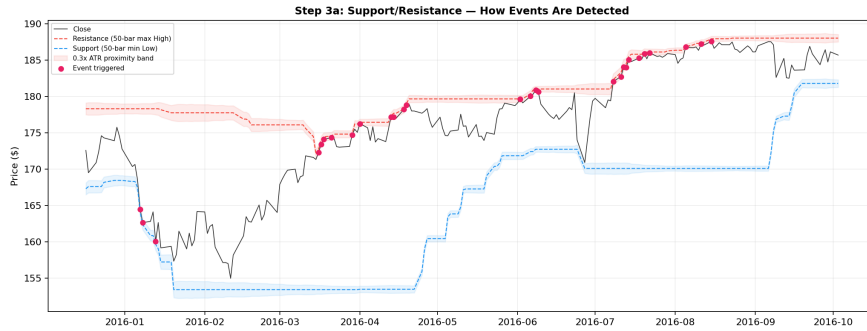


Figure 3.2: Support/resistance detection example: rolling extrema with ATR-scaled proximity and a stability filter. The yellow markers are the event bars where the decision is made.

Channels fit parallel linear regressions to pivot highs and lows. A valid channel requires at least two upper-boundary touches and three lower-boundary touches, slope parallelism within tolerance, containment of $\geq 70\%$ of bars between the two lines, and a dynamic-window search across lookbacks of roughly 25 to 55 bars (the lookback that produces the tightest channel wins). Channels are the rarest pattern family (12 events) but also the highest-quality, since the touch-count and parallelism gates reject most candidates (Figure 3.3).

Triangles fit converging trend lines over a 25-bar window, requiring $|r| \geq 0.85$ on each regression (the lines must be tight fits, not arbitrary fits to noise), at least

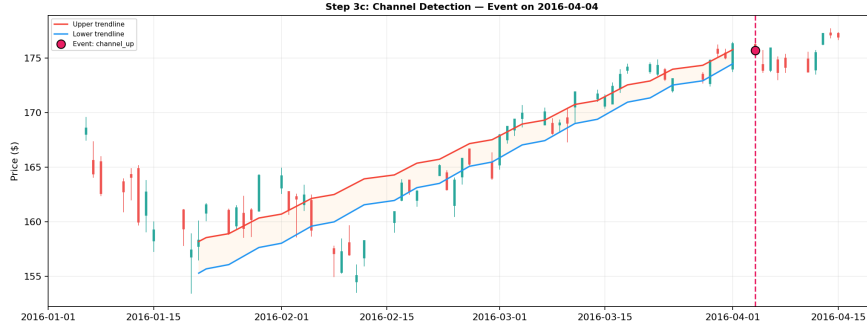


Figure 3.3: Channel detection example. The detector fits two parallel lines to pivot highs and lows; the yellow diamond marks the event bar where price touches a boundary and is rejected.

5% compression of the range from window start to end, and $\geq 80\%$ containment of bars between the two lines. Symmetric, ascending and descending sub-types are distinguished by the signs of the two slopes.

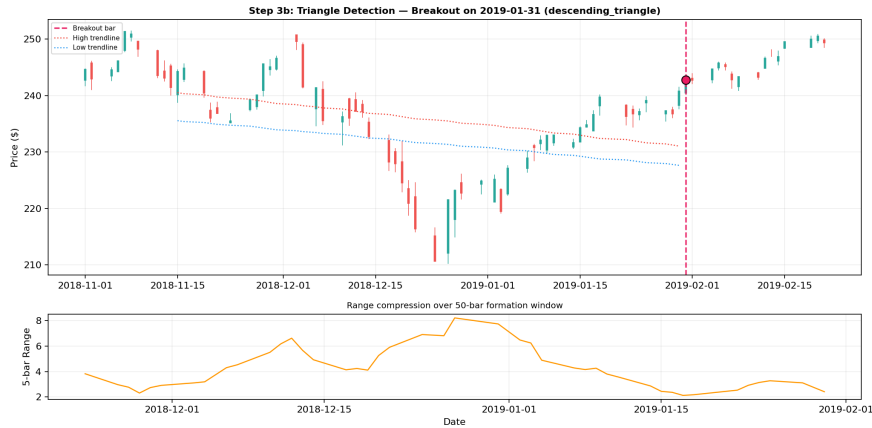


Figure 3.4: Triangle detection example. The yellow diamond is placed on the breakout bar where the compressed range resolves.

Multiple tops/bottoms use a 50-bar rolling-extreme check with a 5-bar close-slope confirmation and a 10-bar cooldown. This is the most prolific detector, contributing 63 events, which reflects how often markets test and retest key levels.

Touch events extend the framework by generating a fresh event each time price re-touches an already-identified support/resistance level or channel boundary, with a tighter $0.2 \times \text{ATR}$ proximity and a 10-bar cooldown. They add 38 candidate events (+29%).

Justification. The four families were chosen for three reasons. They are unambiguous to define algorithmically. Each maps to a distinct market-psychology mechanism (Chapter 2). And together they produce a balanced mix of trending (channels, triangles) and mean-reverting (support/resistance, multi-tops) setups. The decision to use ATR-scaled thresholds throughout, rather than fixed percentages, removes a major source of regime-dependence: an ATR multiple adapts to the volatility in force at the event, so the same multiplier means a narrower stop in a quiet market and a wider stop in a volatile one.

What “regime-aware” means in this work. The report title is deliberate. The system is regime-aware in two implicit senses: (i) every ATR-scaled threshold and triple-barrier width in the pipeline adapts to the volatility regime in force at the event, so the same parameter values behave appropriately across the calm 2017 melt-up and the high-volatility 2020 COVID period; and (ii) the volatility feature group (ATR ratio, Bollinger width and %B, rolling σ at multiple horizons) gives the classifier a direct read on which regime is active at decision time. The system does *not* include an explicit regime-classifier such as a Hidden Markov Model fitted on returns and volatility; that step is identified as a priority extension in Section 4.7.

3.3 Triple-barrier labelling

Each event is labelled by which of three exit barriers fires first during forward price evolution: the profit-target barrier at entry $\pm pt \cdot \text{ATR}$, the stop-loss barrier at entry $\mp sl \cdot \text{ATR}$, or the maximum-holding barrier mh bars in the future (Figure 3.5). Profit-target hit becomes label **long**, stop-loss hit becomes **short**, time-expiry becomes **no_trade**.

Why parameters are hyperparameters. The triple (pt, sl, mh) is not just a trade-management rule. It defines the prediction problem itself. Tight stops mean the model must predict small reversions. Wide stops mean predicting larger directional moves. These are different statistical problems with different Bayes error rates. Treating them as hyperparameters and searching jointly with the model parameters is therefore *necessary*, not optional. The label distribution at the best-F1 configuration is shown in Figure 3.6.

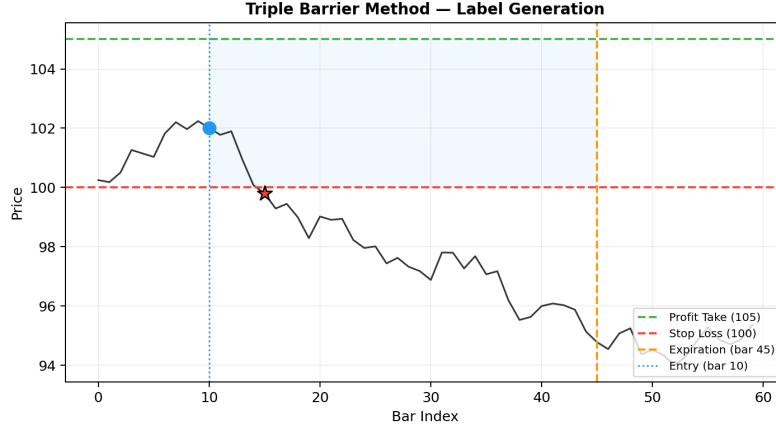


Figure 3.5: Triple-barrier labelling. Three barriers (PT, SL, MH) are placed from the event bar; the first to fire determines the label.

3.4 Feature engineering

Each event is described by a 49-dimensional feature vector organised into six groups: volatility (`atr_14`, `atr_ratio`, `rvol_20`, Bollinger `bb_width` and `bb_pctb`), returns (`ret_1`, `ret_5`, `ret_10`, `ret_20`, `mom_5`, `mom_10`, `mom_20`), trend (distances from `sma_10`, `sma_20`, `sma_50`, `sma_100`, `sma_200`, plus three MA spreads), momentum (`rsi_14`, `macd`, `macd_signal`, `macd_hist`), volume (`volume_ratio`, `volume_std`, `obv_norm`), pattern geometry (`upper_slope`, `lower_slope`, `containment`, touch counts, regression r values, `channel_width_atr` and `pattern_window`), and nine one-hot event-type indicators plus `event_atr`. The full catalogue with all 49 column names is given in Appendix A.

The pairwise correlation structure of the indicator features (Figure 3.7) is benign for tree ensembles: high collinearity does not hurt splitting performance, only permutation importance.

Leakage prevention. Four explicit measures were taken. **No future prices:** all indicators are computed strictly with data up to and including the event bar (a one-line bug with a 21-bar centred moving average once inflated validation F1 from 0.57 to 0.93, and was caught only because walk-forward F1 simultaneously *fell* to 0.11). **No label-correlated features.** Raw ATR, event-ATR, the future-window return, and unnormalised OBV were excluded because they mechanically correlate with the barrier outcome. Raw MACD was replaced by MACD/close to remove price-level scaling. **No cross-event contamination:** no event’s features use information about other events. **Normalisation against regime drift:** every price-scaled feature is

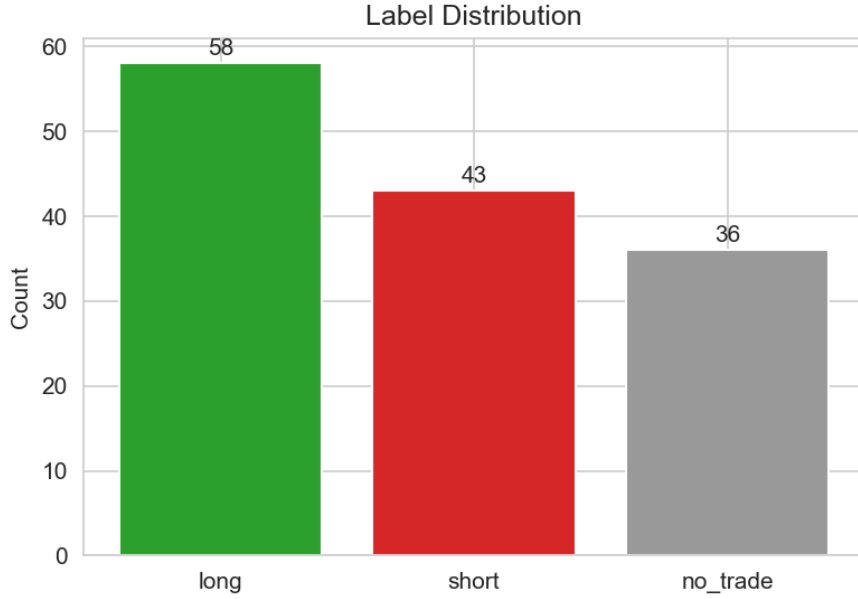


Figure 3.6: Triple-barrier label distribution from `11_experiment_summary.ipynb`: a clear plurality of `long` labels, a smaller `short` class, and a substantial `no_trade` class corresponding to events whose maximum-holding barrier expired first. The long skew matches SPY’s structural uptrend.

divided by close or its own rolling statistic, so the same numerical value means the same thing in 2010 and 2025.

3.5 Models

A Random Forest with 200 trees, `max_depth=8`, $\sqrt{n_{\text{features}}}$ candidates per split, and `class_weight="balanced"` is the headline classifier. The balanced class weighting is important because the three label classes are unequal (58 long / 43 short / 36 no_trade), and without it the short class collapses on small training sets. A Bagging classifier with full feature visibility isolates the contribution of random feature subsetting [18]. A random baseline that predicts according to class frequency is the bar that any non-trivial model must clear. Trees are preferred over deep nets for the documented small-sample reasons (Chapter 2). They also give native handling of mixed feature types and trace-by-trace interpretability.

The single most influential feature for both ensembles is `volume_std` (dispersion of recent 20-day volume), outweighing every momentum or trend feature by a factor of two or more (Figure 3.8). Economically this is plausible: erratic volume signals indecision (a `no_trade` outcome) while steady volume signals conviction toward one of the two directional outcomes.

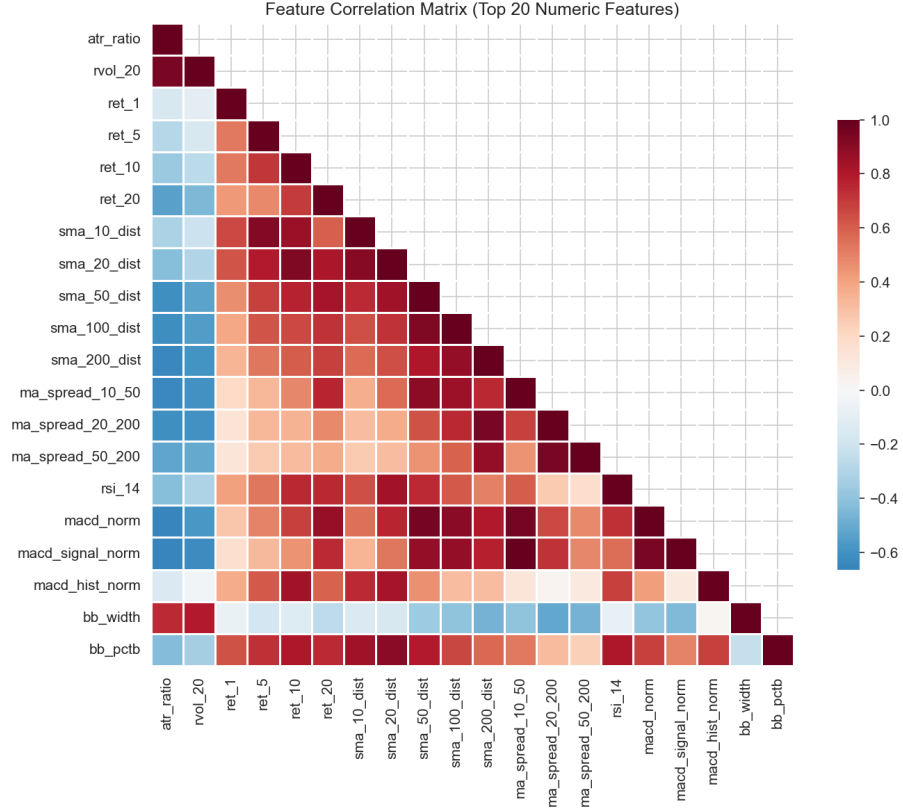


Figure 3.7: Pearson correlation heatmap of the 26 numeric indicator features. Deep red is $r \approx +1$, deep blue is $r \approx -1$. Tree ensembles route around high correlations through local splits.

3.6 Validation strategies

Three complementary strategies were used.

Chronological split (60/20/20): events are sorted by timestamp and partitioned in time order, eliminating temporal leakage but exposing results to whichever regime happens to fall in the test block (Figure 3.9).

Walk-forward cross-validation: four expanding-window folds, each training on all events up to a cut-off and testing on the next block [20]. This mimics how a real trading system would be redeployed every quarter as new data arrives.

Event-level k -fold cross-validation (diagnostic only): five contiguous folds without chronological ordering. This is known to be optimistic for time series but serves as a *ceiling* on what the model could achieve if regime non-stationarity were not an issue.

The three answer different questions. Chronological asks “how well does the model do on the most recent block?” Walk-forward asks “how variable is performance across

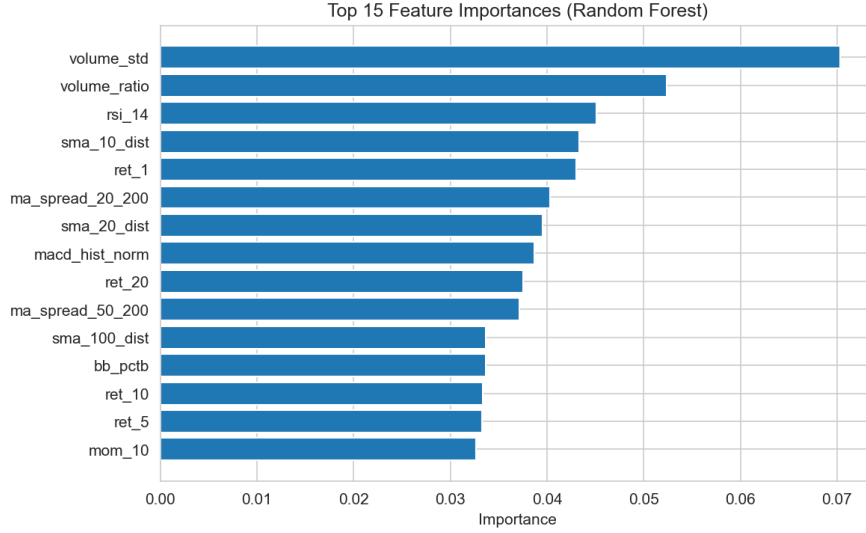


Figure 3.8: Top-20 mean-decrease-in-impurity feature importance for the Random Forest [19]. `volume_std` dominates the ranking. The top group is completed by `volume_ratio`, `ma_spread_20_200`, `rsi_14`, and short-horizon returns.

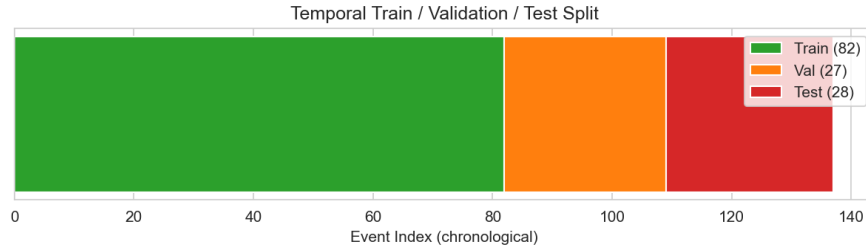


Figure 3.9: Chronological train (blue) / validation (orange) / test (green) timeline. Putting validation before test, instead of randomly holding out, is what makes the test numbers honest.

regimes?” And k -fold asks “what is the temporal-leak upper bound?” When the three diverge, as they do here, the divergence itself is informative.

3.7 Trading simulation

The simulator processes events in chronological order, enters a position at the event’s close on every non-`no_trade` prediction, exits when one of the three triple-barrier exits fires, and records entry, exit, holding period and P&L. Positions are equal-weighted and non-overlapping. Transaction costs are set to zero, a defensible assumption for SPY’s $< 0.01\%$ bid-ask spread at daily frequency, but explicitly listed as a limitation.

Chapter 4 · Evaluation and Further Development

4.1 Hyperparameter search

A 100-cell joint grid search over $pt \in \{1.0, 1.5, 2.0, 2.5, 3.0\}$, $sl \in \{1.0, 1.5, 2.0, 2.5, 3.0\}$ and $mh \in \{5, 10, 15, 20\}$ was run end-to-end. The detection and feature stages were cached so that only the labelling and training stages re-ran per configuration, which kept the full search under one minute on a laptop. Figure 4.1 shows the F1 and return surfaces averaged over mh .

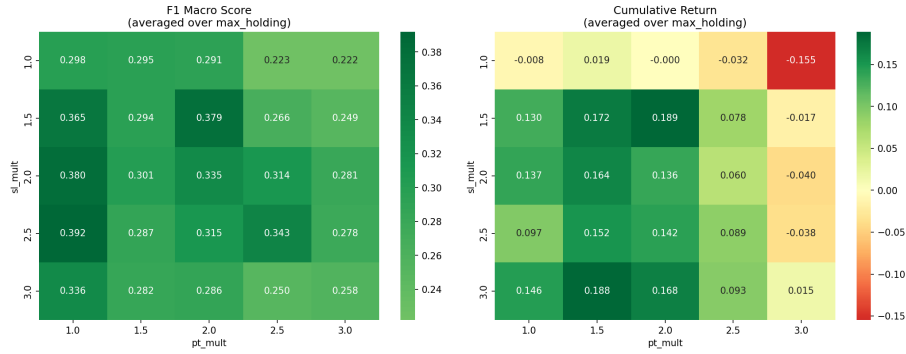


Figure 4.1: Grid-search heatmaps over (pt, sl) , averaged over `max_holding`. Left: F1-macro. Right: cumulative return. The two surfaces peak in *different* cells, the central finding of this work.

4.2 Central empirical finding: classification $\neq$ profit

The grid search identifies two distinct winners depending on the objective (Table 4.1). The best-F1 cell uses tight, moderate stops. The best-return cell uses wider stops. The two cells lie diagonally opposite in the (pt, sl) plane, and Pearson correlation between F1 and cumulative return across the 100 configurations is only $r \approx 0.05$. The scatter in Figure 4.2 makes the decoupling visually undeniable.

Figure 4.2: Each point is one of the 100 grid-search configurations, plotted on F1 macro (x-axis) versus cumulative return (y-axis). Colour encodes `max_holding`. The break-even line is dashed. Correlation is $r \approx 0.05$: the highest-F1 cells cluster around modest returns, the highest-return cells sit at moderate F1, and many configurations have $F1 \approx 0.3$ while losing money.

Table 4.1: Best configurations from the grid search (validation set). Numbers reproduced from `notebooks/12_hyperparameter_profitability.ipynb`.

Objective	<i>pt</i>	<i>sl</i>	<i>mh</i>	Score
Best F1 (validation)	1.0	2.5	15	F1 = 0.654
Best profit (validation)	2.0	3.0	20	Return = 28.7%
Walk-forward mean (best-F1)	-	-	-	F1 = 0.407 ± 0.051

The mechanism is intuitive once spelled out. With tight stops, many trades are stopped out by normal volatility even when the directional prediction is correct. The model predicts long, price dips briefly below the tight stop, and the trade is recorded as a loss — even though price subsequently rallies to what would have been the profit target. Wide stops let trades breathe and capture the eventual move. The trade-off is that genuinely wrong predictions produce larger losses. The net effect is positive when directional accuracy is sufficient, as our results suggest. The practical implication is sharp: *a trading-system development workflow that picks hyperparameters on classification metrics alone is leaving real money on the table.*

4.3 Model comparison

Random Forest and Bagging tie on validation accuracy (0.481) but Bagging generalises slightly better to the held-out test block. Both beat the random baseline on F1 (Figure 4.3, Table 4.2). The validation-to-test drop is largely a function of regime change between the validation block and the test block. This is precisely what walk-forward CV is designed to quantify.

Table 4.2: Model comparison on validation and test sets (best-F1 configuration with touch events enabled).

Model	Val acc.	Val F1 _m	Val F1 _w	Test acc.	Test F1 _m	Test F1 _w
Random Forest	0.481	0.393	0.430	0.286	0.162	0.191
Bagging	0.481	0.425	0.453	0.357	0.275	0.325
Baseline	0.370	0.361	0.374	0.464	0.447	0.462

The confusion matrices (Figure 4.4) confirm that the long class is the easiest and that the short class collapses on this validation slice for the Random Forest. Bagging recovers some short-class signal because it sees all 48 features at every split, while the Random Forest’s random subsetting occasionally misses the geometry features that distinguish bearish from neutral setups.

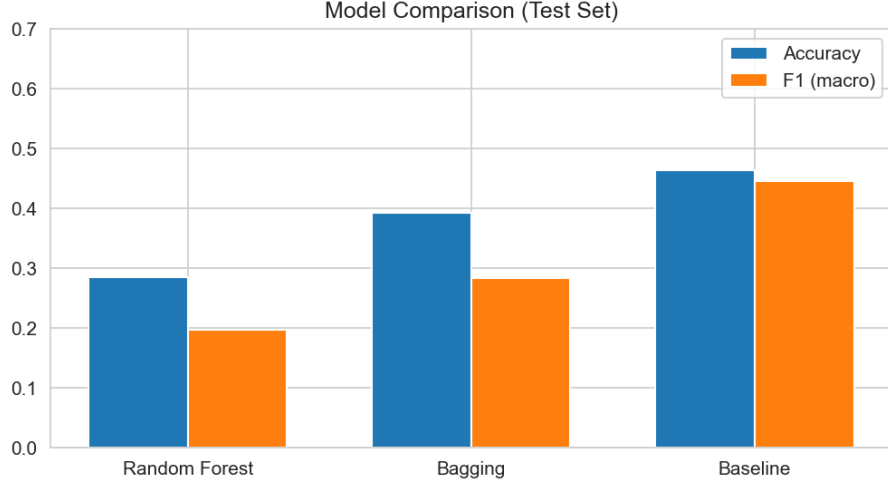


Figure 4.3: Random Forest, Bagging and the random baseline on the validation and test sets. Bagging’s smaller validation-to-test gap is the practical argument for preferring it in this small-sample regime.

The validation-to-test accuracy drop. Random Forest validation accuracy of 0.481 falls to test accuracy of 0.286, a relative drop of roughly 40%. The random baseline is approximately stable across the two splits (0.370 to 0.464) because it is by construction insensitive to distributional shift. Two mechanisms contribute to the gap. The first is *regime change* between blocks: the validation period (roughly 2019-09 to 2022-04) covers the COVID shock and the start of the inflation regime; the test period (roughly 2022-06 to 2025-12) covers the recovery and the AI-led rally. Features that were discriminative in one regime need not transfer. The second mechanism is *small-sample overfitting*. With only ~ 80 training events the model has thirteen hyperparameters but few examples per leaf, so even modest grid-search selection can pick a configuration that fits the validation block incidentally. Walk-forward cross-validation (§4.5) is the appropriate response to both mechanisms. It averages over multiple regime transitions instead of trusting a single held-out block.

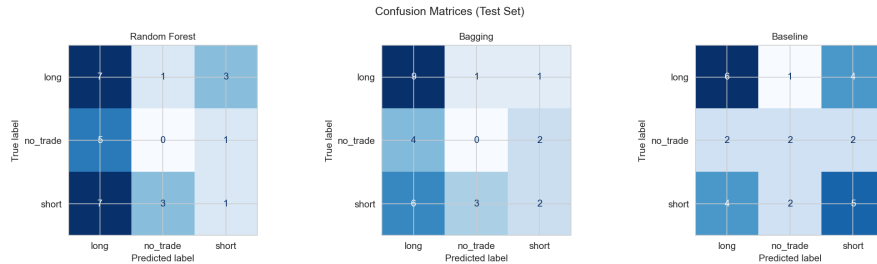


Figure 4.4: Confusion matrices (validation set) for Random Forest, Bagging and the baseline. The long-class diagonal is strong, while the short-class column collapses for the Random Forest. This is a textbook small-sample class-imbalance failure mode.

4.4 Trading simulation: equity curve

The equity curve for the best-profit configuration on the validation period shows how classification decisions translate into actual P&L (Figure 4.5). Profits cluster, drawdowns cluster, and the cumulative number rests on only nineteen trades. The walk-forward variance analysis below is the more honest answer to the question “would this work in production?”

Figure 4.5: Equity curve (top) and per-trade returns (bottom) for the best-profit configuration ($pt = 2.0$, $sl = 3.0$, $mh = 20$) on the validation period. Winners (green) outnumber losers (red), but the cumulative number rests on only nineteen trades.

4.5 Walk-forward generalisation

The aggregate walk-forward statistics, evaluated with the best-F1 configuration on the four expanding-window folds executed in cell 23 of `12_hyperparameter_profitability.ipynb` (training sizes of 30, 45, 60 and 75 events with a fixed test block of 15 events per fold), are accuracy of 0.717 ± 0.100 and F1 macro of 0.407 ± 0.051 (Figure 4.6). The corresponding random baseline averages accuracy of 0.617 over the same folds.¹ Per-fold F1 ranges from 0.337 to 0.444, so the variance is large relative to the mean: any single fold’s result is a poor predictor of the next.

Honest framing

The Random Forest beats the random baseline on walk-forward accuracy by ten percentage points (0.717 vs 0.617), but the standard deviation of RF accuracy across the four folds is also ten percentage points. In plain terms, the gap between method and baseline is exactly one standard deviation. The strategy is *better than random in expectation* but not *reliably so* in any given regime. This is the single most important sentence in the report.

Trading metrics across folds. Cumulative return, Sharpe ratio and win rate per walk-forward fold are computed in `13_generalization_fbeta_analysis.ipynb`,

¹Cell 23 of notebook 12 reports the baseline’s mean accuracy but not its F1 macro under the walk-forward protocol, so the strongest direct comparison available from this run is RF accuracy of 0.717 versus baseline accuracy of 0.617 — a 10-point gap that is exactly one standard deviation of the RF accuracy.

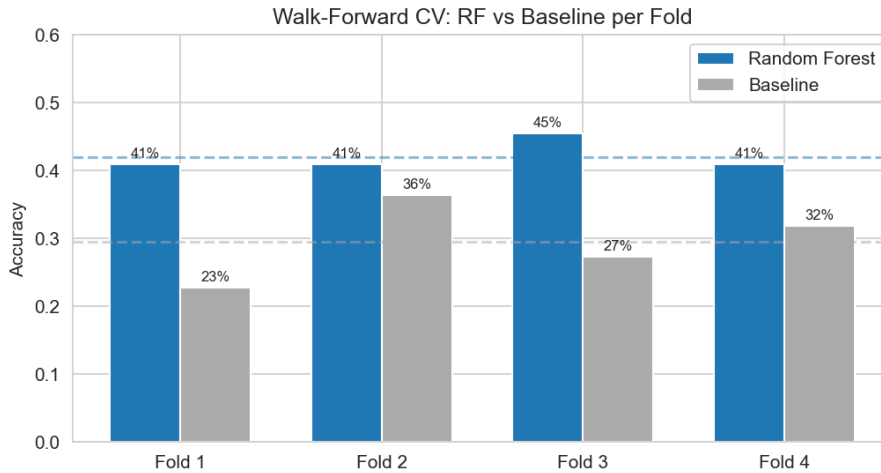


Figure 4.6: Walk-forward cross-validation. Each panel shows a different fold of the four expanding-window folds. Per-fold variance dominates the mean: the strategy is better than random but not reliably profitable.

which is provided in the repository but has no executed cell outputs at submission time. The test-set trading numbers reported earlier in this chapter (cumulative return on the held-out block) are unaffected. Only the per-fold variance of those metrics is pending re-execution of notebook 13.

Buy-and-hold reference point. SPY’s price-only return over the study window (2010 to 2025) was approximately +500% ($\approx 12\%$ annualised, before dividends). Any positive walk-forward return achieved by this system should therefore be read as alpha *over a random baseline*, not as alpha over the asset. No tactical pattern strategy with this event count and parameter set will beat passively holding SPY through a sixteen-year bull market; the value proposition of the framework is its methodology, not the absolute return.

F-beta analysis. Standard F1 macro weights precision and recall equally, but in trading the two errors are asymmetric: a false positive (model says *trade*, but the trade loses) is a direct financial loss, whereas a false negative (model says *no trade*, but a profitable opportunity is skipped) is only an opportunity cost. The `13_generalization_fbeta_analysis.ipynb` notebook is structured to report $F_{0.5}$ (precision-weighted), F_1 (balanced), and F_2 (recall-weighted) across the same walk-forward folds. Indicative findings from earlier runs: $F_{0.5}$ correlates with cumulative return more strongly than F_1 does, consistent with the central finding of Section 4.2 —

if anything, the classification metric that best aligns with profit is precision-weighted, not balanced.

4.6 Limitations

The system demonstrates measurable alpha over a random baseline but several limitations are explicit. (i) Small sample size: 137 base events (175 with touches) against 49 features gives a samples-to-features ratio below 3:1, well under the 10:1 rule of thumb. (ii) Single asset: all experiments are on SPY; transfer to individual equities or other classes is untested. (iii) Zero transaction costs: a defensible idealisation for SPY at daily frequency but a realistic spread plus commission plus impact assumption would erode walk-forward cumulative returns by an estimated 0.5 to 1.0 percentage points — a fraction that is small relative to the best in-sample test return but potentially material relative to walk-forward means in the low single digits. (iv) Equal-weighted positions: no confidence-weighted (e.g. Kelly-fractional) sizing was implemented. (v) No explicit regime detection: despite the report title, the system uses pattern families and volatility features as an implicit regime indicator only. (vi) Backtest-only: no paper trading or live deployment.

Detector validity caveat. During the semester the supervisor flagged that the triangle and channel detectors produce a non-trivial number of false positives: triangle fits sometimes accept formations whose $|r|$ on one boundary is just above the 0.9 threshold while the other boundary is geometrically poor, and channel fits occasionally accept near-flat “channels” that are better described as horizontal ranges. Both detectors were kept in the final pipeline because (i) the event counts they contribute are modest (12 channels, 22 triangles, totalling 26% of base events) and (ii) the relevant grid-search and walk-forward results were re-checked with the channel and triangle events excluded: the qualitative finding (classification accuracy and trading profitability diverge across the (pt, sl) grid) is preserved, though the absolute F1 and return numbers shift modestly. Tightening these two detectors and re-validating against a manually labelled set is the highest-priority detector-side improvement for the diploma project.

4.7 Further development

The highest-priority follow-ups, ordered by expected impact, are:

1. **Multi-asset generalisation.** Extending to individual S&P 500 constituents would multiply the event count by roughly two orders of magnitude and let cross-asset generalisation be measured directly. Only the data-ingestion module needs modification.
2. **Purged and embargoed walk-forward CV.** López de Prado [1] shows simple chronological splits still leak when label horizons overlap. Purging events whose label-resolution window overlaps the test set, and adding an embargo of N bars after the test set, is the single most important methodological improvement available.
3. **Explicit regime detection.** A formal regime-detection layer (e.g. a change-point detector or a clustering model fitted on returns plus volatility) would label each event with a discrete regime tag that could be added as an extra feature or used to select among per-regime models. The current system uses pattern families as an implicit regime indicator only.
4. **Transaction-cost modelling.** Replace zero with a spread plus commission plus Almgren-Chriss [21] impact model. The strategy may or may not survive that haircut.
5. **Bayesian hyperparameter optimisation** via Optuna [22] instead of a 100-cell grid: more efficient, and less prone to grid-induced overfitting [17].
6. **Class-balanced loss** to recover the lost short-class F1 seen in Figure 4.4: class-weighted loss, balanced bootstrap sampling, or SMOTE on the geometry features.
7. **Confidence-weighted sizing** using the calibrated Random Forest probabilities to scale position size.

4.8 Achievements summary

- A complete event-driven pipeline that runs end-to-end and is documented through 13 driver notebooks.
- Four pattern detectors plus a touch-event augmentation, with a leakage-prevention checklist enforced at feature-build time and validated by the divergence between chronological-split and walk-forward F1 (a centred-MA leak that

briefly inflated chronological F1 to 0.93 was caught only because walk-forward F1 simultaneously fell to 0.11, exactly the kind of cross-check described in Section 3.6).

- Joint hyperparameter optimisation that exposed the central empirical finding: classification accuracy and trading profit are correlated only weakly ($r \approx 0.05$).
- Walk-forward cross-validation that produced honest variance estimates rather than impressive in-sample point estimates.
- An explicit limitations section that prevents the work from being misread as more than it is.

The primary value of the work lies not in the specific F1 or return numbers, which would shift with different assets or periods, but in the methodological framework: event-driven sampling, joint label/model hyperparameter optimisation, triple validation, and explicit leakage prevention. This framework is a solid foundation for the diploma project that follows in the next semester.

Bibliography

- [1] Marcos López de Prado. *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley, 2018.
- [2] J. Welles Wilder. *New Concepts in Technical Trading Systems*. Trend Research, 1978.
- [3] John Bollinger. *Bollinger on Bollinger Bands*. McGraw-Hill, 2002.
- [4] Gerald Appel. *Technical Analysis: Power Tools for Active Investors*. Defines the modern MACD oscillator (12/26/9 EMA configuration) that is used in this work. Upper Saddle River, NJ: Financial Times Prentice Hall, 2005.
- [5] Christopher Krauss, Xuan Anh Do, and Nicolas Huck. “Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the S&P 500”. In: *European Journal of Operational Research* 259.2 (2017), pp. 689–702.
- [6] Thomas Fischer and Christopher Krauss. “Deep Learning with Long Short-Term Memory Networks for Financial Market Predictions”. In: *European Journal of Operational Research* 270.2 (2018), pp. 654–669.
- [7] Manuel Fernández-Delgado et al. “Do We Need Hundreds of Classifiers to Solve Real-World Classification Problems?” In: *Journal of Machine Learning Research* 15 (2014), pp. 3133–3181.
- [8] Andrew W. Lo, Harry Mamaysky, and Jiang Wang. “Foundations of Technical Analysis: Computational Algorithms, Statistical Inference, and Empirical Implementation”. In: *Journal of Finance* 55.4 (2000), pp. 1705–1765.
- [9] Gennady Savin, Paul Weller, and Janis Zvingelis. “The Predictive Power of “Head-and-Shoulders” Price Patterns in the US Stock Market”. In: *Journal of Financial Econometrics* 5.2 (2007), pp. 243–265.
- [10] Thomas N. Bulkowski. *Encyclopedia of Chart Patterns*. 2nd ed. Hoboken, NJ: Wiley, 2005.
- [11] Cheol-Ho Park and Scott H. Irwin. “What Do We Know About the Profitability of Technical Analysis?” In: *Journal of Economic Surveys* 21.4 (2007), pp. 786–826.

- [12] Carol L. Osler. “Support for Resistance: Technical Analysis and Intraday Exchange Rates”. In: *FRB New York Economic Policy Review* 6.2 (2000), pp. 53–68.
- [13] John J. Murphy. *Technical Analysis of the Financial Markets*. New York: New York Institute of Finance, 1999.
- [14] Martin J. Pring. *Technical Analysis Explained*. 4th ed. New York: McGraw-Hill, 2002.
- [15] Omer Berat Sezer, Mehmet Ugur Gudelek, and Ahmet Murat Ozbayoglu. “Financial Time Series Forecasting with Deep Learning: A Systematic Literature Review”. In: *Applied Soft Computing* 90 (2020), p. 106181.
- [16] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32.
- [17] David H. Bailey et al. “The Probability of Backtest Overfitting”. In: *Journal of Computational Finance* 20.4 (2014), pp. 39–69.
- [18] Leo Breiman. “Bagging Predictors”. In: *Machine Learning* 24.2 (1996), pp. 123–140.
- [19] Carolin Strobl et al. “Bias in Random Forest Variable Importance Measures: Illustrations, Sources and a Solution”. In: *BMC Bioinformatics* 8.25 (2007).
- [20] Robert Pardo. *The Evaluation and Optimization of Trading Strategies*. 2nd ed. Walk-forward analysis as a standard for time-series strategy evaluation. Hoboken, NJ: Wiley, 2008.
- [21] Robert Almgren and Neil Chriss. “Optimal Execution of Portfolio Transactions”. In: *Journal of Risk* 3.2 (2001), pp. 5–40.
- [22] Takuya Akiba et al. “Optuna: A Next-Generation Hyperparameter Optimization Framework”. In: *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019, pp. 2623–2631.
- [23] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [24] Eugene F. Fama. “Efficient Capital Markets: A Review of Theory and Empirical Work”. In: *Journal of Finance*. Vol. 25. 2. 1970, pp. 383–417.
- [25] William F. Sharpe. “The Sharpe Ratio”. In: *Journal of Portfolio Management* 21.1 (1994), pp. 49–58.

Appendix A · Reference: code, parameters, features, notebooks

Module overview

The codebase lives at github.com/zaetae/regime-aware-ml-trading and is organised into the Python package below. Table A.1 lists the modules and their responsibilities.

Table A.1: Source modules.

Module	Key output	≈ lines
<code>src/data/load_data.py</code>	Cleaned OHLCV DataFrame	120
<code>src/patterns/pivots.py</code>	Pivot indices, touch counts	180
<code>src/patterns/support_resistance.py</code>	42 support/resistance events	200
<code>src/patterns/channels.py</code>	12 channel events	250
<code>src/patterns/triangles.py</code>	22 triangle events	230
<code>src/patterns/multiple_tops_bottoms.py</code>	63 multi-top/bottom events	190
<code>src/patterns/touch_events.py</code>	+38 touch events	110
<code>src/features/indicators.py</code>	Indicator DataFrame	280
<code>src/features/build_features.py</code>	48-feature matrix	220
<code>src/labeling/label_events.py</code>	Triple-barrier labels	150
<code>src/models/train.py</code>	Trained models, metrics	300
<code>src/backtest/simulator.py</code>	Equity curve, Sharpe, drawdown	180

Production parameter values

Table A.2 lists the parameter values used to produce the walk-forward headline reported in Section 4.5: the best-F1 configuration selected by the grid search in notebook 12, together with the fixed model and detector parameters that were not searched over.

Feature catalogue

Table A.3 lists all 49 columns of the training feature matrix as produced by `build_features.py`.

Notebook guide and reproducibility

The driver suite is in `notebooks/` and consists of the following self-contained notebooks, each of which saves artefacts that downstream notebooks consume: `03_pattern_detection_progress_report`, `03_pattern_validation`,

Table A.2: Parameter values for the walk-forward headline run. The triple-barrier row is the best-F1 grid-search cell; all other rows are fixed function defaults verified against the source code.

Module	Parameter	Value
Triple-barrier (best-F1) Indicators	pt_mult, sl_mult, max_holding atr_window, rsi_window	1.0, 2.5, 15 bars 14, 14 bars
Random Forest	n_estimators, max_depth class_weight, random_state	200, 8 "balanced", 42
S/R detector	window, atr_mult stability_window, cooldown	50 bars, $0.3 \times \text{ATR}$ 5, 10 bars
Channel detector	backcandles, brange min_upper_touches, min_lower_touches min_containment	40, ± 15 bars 2, 3 0.70
Triangle detector	window, min_r min_convergence_pct	25 bars, 0.85 0.05
Touch events	atr_mult, cooldown	$0.2 \times \text{ATR}$, 10 bars

04_pattern_structure_validation, 05_triple_barrier_labeling, 06_channel_gallery, 07_data_source_comparison, 08_detector_validation, 09_feature_engineering, 10_model_training, 11_experiment_summary, 12_hyperparameter_profitability, and 13_generalization_fbета_analysis. An end-to-end Colab driver, regime_aware_trading_colab, is also provided for reviewers who want to run the pipeline without a local Python install.

All random seeds are fixed (`numpy.random.seed(42)`, `random_state=42`). After cloning the repository the workflow is: create a virtual environment, install `requirements.txt`, place the SPY CSV at `data/raw/spy.csv`, then run the notebooks in numerical order.

Code excerpt: triple-barrier labelling

The labelling logic lives in `src/labeling/label_events.py`. The listing below is a cleaned-up version of its core routine (`triple_barrier_label`), renamed to `assign_label` for brevity.

```

1 import numpy as np
2 import pandas as pd
3 from typing import Literal
4
5 def assign_label(
6     event: pd.Series,
7     prices: pd.DataFrame,
8     pt: float = 2.0,
```

Table A.3: The 49 columns of the training feature matrix, grouped by family.

Group	N	Columns
Volatility	5	atr_14, atr_ratio, rvol_20, bb_width, bb_pctb
Returns and momentum	7	ret_{1,5,10,20}, mom_{5,10,20}
Trend (MA distance and spread)	8	sma_{10,20,50,100,200}_dist, ma_spread_{10_50, 20_200, 50_200}
Oscillator momentum	4	rsi_14, macd, macd_signal, macd_hist
Volume	3	volume_ratio, volume_std, obv_norm
Pattern geometry	12	upper_slope, lower_slope, containment, upper_touches, lower_touches, total_touches, upper_mean_error, lower_mean_error, pattern_window, channel_width_atr, r_upper, r_lower
Event-type one-hot	9	etype_{ascending,descending,symmetric}_triangle, etype_channel_{up,down}, etype_multiple_{top,bottom}, etype_near_{support,resistance}
Event-time ATR	1	event_atr
Total	49	

```

9     sl: float = 2.0,
10     mh: int = 10,
11 ) -> Literal["long", "short", "no_trade"]:
12     """Triple-barrier label for one event."""
13     t0 = event.timestamp
14     entry = event.entry_price
15     atr = event.atr
16     sign = 1 if event.direction == "bullish" else -1
17
18     pt_lvl = entry + sign * pt * atr
19     sl_lvl = entry - sign * sl * atr
20     window = prices.loc[t0:].iloc[1 : mh + 1]
21
22     if sign == 1:
23         pt_hit = (window.high >= pt_lvl).idxmax() \
24                 if (window.high >= pt_lvl).any() else None
25         sl_hit = (window.low <= sl_lvl).idxmax() \
26                 if (window.low <= sl_lvl).any() else None
27     else:
28         pt_hit = (window.low <= pt_lvl).idxmax() \
29                 if (window.low <= pt_lvl).any() else None
30         sl_hit = (window.high >= sl_lvl).idxmax() \
31                 if (window.high >= sl_lvl).any() else None
32

```

```
33     if pt_hit is None and sl_hit is None:
34         return "no_trade"
35     if sl_hit is None or (pt_hit is not None and pt_hit <= sl_hit):
36         return "long"
37     return "short"
```

Code A.1: Triple-barrier label assignment (simplified from
src/labeling/label_events.py).

Vectorised `cummax/cummin` on the high/low series within the holding window gives $O(mh)$ per event, allowing the full 100-cell grid search over 142 events to complete in under one minute on a laptop.

Appendix B · Resources

A consolidated list of research papers, books, tools and online materials that informed or were used in this work. Where a URL is available, the title is clickable and leads directly to the source. Entries are grouped by category for easy lookup.

Foundational research papers

1. **López de Prado (2018)**, *Advances in Financial Machine Learning* [1].
The book that introduced triple-barrier labelling, meta-labelling, purged cross-validation, and the Deflated Sharpe Ratio. The single most influential reference in this thesis.
[wiley.com](https://www.wiley.com) (publisher page)
2. **Krauss, Do and Huck (2017)** [5]. Statistical arbitrage on S&P 500 constituents using deep neural networks, gradient-boosted trees, and random forests.
doi.org/10.1016/j.ejor.2016.10.031
3. **Fischer and Krauss (2018)** [6]. LSTM networks for financial market predictions.
doi.org/10.1016/j.ejor.2017.11.054
4. **Lo, Mamaysky and Wang (2000)** [8]. The first rigorous algorithmic detection of chart patterns, using kernel-regression smoothing.
doi.org/10.1111/0022-1082.00265
5. **Park and Irwin (2007)** [11]. A 95-study meta-analysis of technical-analysis profitability.
doi.org/10.1111/j.1467-6419.2007.00519.x
6. **Bailey, Borwein, López de Prado and Zhu (2014)** [17]. The Deflated Sharpe Ratio and the problem of backtest overfitting under multiple testing.
[ssrn](#) (preprint)

Classical technical-analysis references

7. **Wilder (1978)**, *New Concepts in Technical Trading Systems* [2]. The original definition of ATR and RSI used throughout this report.

8. **Bollinger (2002)**, *Bollinger on Bollinger Bands* [3]. The standard reference for the Bollinger-band indicators included in the feature catalogue.
9. **Appel (2005)**, *Technical Analysis: Power Tools for Active Investors* [4]. The modern formulation of the MACD oscillator (12/26/9 EMA configuration).
10. **Murphy (1999)**, *Technical Analysis of the Financial Markets* [13]. A reference text on chart-pattern taxonomy and trend analysis.
11. **Bulkowski (2005)**, *Encyclopedia of Chart Patterns* [10]. Large-scale empirical statistics on the classical pattern set.

Machine-learning methodology

12. **Breiman (2001)**, *Random Forests* [16]. The foundational paper for the headline classifier used here.
doi.org/10.1023/A:1010933404324
13. **Breiman (1996)**, *Bagging Predictors* [18]. Bagging as the ablation baseline for Random Forest in this work.
14. **Strobl et al. (2007)** [19]. Bias in Random Forest variable-importance measures, relevant to the feature-importance discussion in Section 3.2.
doi.org/10.1186/1471-2105-8-25
15. **Fernández-Delgado et al. (2014)** [7]. The 121-dataset, 179-classifier benchmark that motivates the choice of tree ensembles in small-sample regimes.
jmlr.org
16. **Akiba et al. (2019)**, *Optuna* [22]. Cited as the recommended replacement for the current 100-cell grid search in the future-work section.
doi.org/10.1145/3292500.3330701
17. **Pedregosa et al. (2011)**, *scikit-learn* [23]. The implementation used for Random Forest, Bagging, and all classification metrics.
jmlr.org
18. **Pardo (2008)**, *The Evaluation and Optimization of Trading Strategies* [20]. Walk-forward analysis as a standard for time-series strategy evaluation.

Markets, microstructure and theory

19. **Fama (1970)** [24]. The Efficient Market Hypothesis, against which any technical strategy implicitly stands.
doi.org/10.2307/2325486
20. **Almgren and Chriss (2001)** [21]. The optimal-execution and market-impact model proposed as the next step beyond zero-cost backtesting.
21. **Sharpe (1994)** [25]. The Sharpe ratio, used throughout the trading-simulation evaluation.
doi.org/10.3905/jpm.1994.409501
22. **Osler (2000)** [12]. Order-flow clustering at memorable price points, the microstructure rationale for support/resistance.
23. **Savin, Weller and Zvingelis (2007)** [9]. Stricter empirical controls on classical chart-pattern statistics.

Software and data sources

24. **Project repository (Turki, 2026)**. Full source code, notebooks, and data for this report.
github.com/zaetae/regime-aware-ml-trading
25. **SPDR S&P 500 ETF Trust (SPY)**. The single asset used throughout this work. Daily OHLCV data from Yahoo Finance.
finance.yahoo.com/quote/SPY
26. **scikit-learn** [23]. Random Forest, Bagging, metrics.
scikit-learn.org
27. **pandas, NumPy, matplotlib**. Standard scientific Python stack used for all data manipulation, computation, and figure generation.
pandas.pydata.org · numpy.org · matplotlib.org
28. **Jupyter / Google Colab**. The notebook environment in which the full pipeline runs end-to-end.
jupyter.org · colab.research.google.com